

BK Restaurant System
Issue 13: System Architecture Design
Architectural Decision Record (ADR)

ADR-001: Adopt Three-Tier Modular Architecture

Date: 2026-06-11

Status: Accepted

1. Context

The BK Restaurant Management System must support multiple modules including POS operations, inventory management, delivery tracking, employee management, and analytics across three restaurant branches.

The system must satisfy maintainability, scalability, security, and multi-branch operation requirements while allowing future enhancements.

Relevant Requirements:

- NFR-16 Modular Architecture
- NFR-14 Branch Expansion
- NFR-15 Load Scalability
- FR-15 Multi-Branch Monitoring Dashboard

2. Decision

Adopt a Three-Tier Modular Architecture consisting of:

1. Presentation Layer (React Web Application + Flutter Mobile App)
2. Business Logic Layer (Node.js/Express REST API)
3. Data Layer (PostgreSQL Database)

All communication will occur through REST APIs.

3. Rationale

This architecture separates responsibilities clearly and improves maintainability.

Benefits:

- Easier future enhancements
- Independent frontend/backend development
- Better scalability
- Simplified testing
- Supports multi-branch operations

The design directly supports scalability and maintainability requirements defined in the SRS.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
Monolithic Application	Easy initial development	Difficult to maintain and scale	Not suitable for future growth
Microservices Architecture	Highly scalable	Complex deployment and management	Too complex for project scope
Three-Tier Modular Architecture	Good scalability and maintainability	Slightly more setup effort	Selected

5.Consequences

Positive

- Improved maintainability
- Better scalability
- Clear separation of concerns
- Easier future module additions

Negative

- More components to manage
- Requires API integration between layers

6.References

- NFR-16 Modular Architecture
- NFR-14 Branch Expansion
- NFR-15 Load Scalability
- Section 2.4 Design Constraints in SRS Document

ADR-002: Use React.js for Web Frontend

Date: 2026-06-11

Status: Accepted

1.Context

The system requires responsive web interfaces for:

- POS operations
- Inventory management
- Employee management

- Analytics dashboard

The user interface must be easy for non-technical staff to use.

Relevant Requirements:

- NFR-08 Simple UI
- NFR-09 Clear Navigation
- NFR-19 Browser Support

2.Decision

Use React.js with Tailwind CSS for the web frontend.

3.Rationale

React provides:

- Component reusability
- Fast user interface updates
- Strong ecosystem support
- Responsive design capabilities

Tailwind CSS allows rapid development of consistent interfaces.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
Angular	Complete framework	Steeper learning curve	More complex than required
Vue.js	Lightweight	Smaller ecosystem	Less team familiarity
React.js	Large ecosystem, reusable components	Requires component management	Selected

5.Consequences

Positive

- Faster UI development
- Reusable components
- Responsive interfaces

Negative

- Initial learning curve
- State management complexity

6.References

- Section 2.3 Operating Environment in SRS Document
- NFR-08
- NFR-09
- NFR-19

ADR-003: Use Node.js and Express.js for Backend Services

Date: 2026-06-11

Status: Accepted

1.Context

The backend must support:

- Order processing
- Inventory management
- Delivery tracking
- Employee management
- Analytics generation

The system requires real-time processing and concurrent access.

Relevant Requirements:

- NFR-02 Concurrent Access
- NFR-03 Real-Time Processing
- FR-01 to FR-18

2.Decision

Implement backend services using Node.js and Express.js.

3.Rationale

Node.js provides:

- Event-driven architecture
- High concurrency support
- Fast API response times
- Large package ecosystem

Express.js simplifies REST API development.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
Spring Boot	Enterprise-grade	More complex setup	Overkill for project scope
ASP.NET Core	High performance	Requires C# expertise	Team familiarity lower
Node.js + Express	Lightweight, scalable, JavaScript ecosystem	Single-threaded limitations	Selected

5.Consequences

Positive

- Fast API development
- Good performance
- Shared JavaScript ecosystem

Negative

- Requires proper error handling
- CPU-intensive tasks need optimization

6.References

- Section 2.3 Operating Environment in SRS Document
- NFR-02
- NFR-03

ADR-004: Use PostgreSQL as Primary Database

Date: 2026-06-11

Status: Accepted

1.Context

The system stores:

- Orders
- Inventory
- Employees
- Attendance
- Deliveries

- Financial records

Data consistency and reliability are critical.

Relevant Requirements:

- NFR-12 Data Consistency
- NFR-13 Backup & Recovery
- FR-01, FR-09, FR-16

2.Decision

Use PostgreSQL as the primary relational database.

3.Rationale

PostgreSQL provides:

- ACID compliance
- Strong relational support
- Data integrity
- Excellent reporting capabilities

These are important for restaurant transactions and inventory records.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
MongoDB	Flexible schema	Weaker relational modeling	Not ideal for transactional data
MySQL	Popular and reliable	Slightly fewer advanced features	PostgreSQL preferred
PostgreSQL	Strong consistency and reliability	Slightly more configuration	Selected

5.Consequences

Positive

- Reliable transaction handling
- Strong reporting support
- Data consistency

Negative

- More complex administration than NoSQL databases

6.References

- Section 2.3 Operating Environment in SRS Document
- NFR-12
- NFR-13

ADR-005: Use Flutter for Delivery Rider Mobile Application

Date: 2026-06-11

Status: Accepted

1.Context

Delivery riders require a mobile application for:

- Receiving assignments
- GPS navigation
- Delivery status updates

Relevant Requirements:

- FR-06 Rider Mobile Application
- FR-07 GPS-Based Delivery Tracking
- FR-08 Delivery Status Updates
- NFR-10 Rider App Usability
- NFR-20 Android Support

2.Decision

Develop the rider mobile application using Flutter.

3.Rationale

Flutter provides:

- Single codebase
- Native-like performance
- Strong GPS integration support
- Fast UI development

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
Native Android	High performance	Separate development effort	Less efficient
React Native	Cross-platform	Dependency on native bridges	Less preferred
Flutter	Single codebase and strong UI support	Larger application size	Selected

5.Consequences

Positive

- Faster development
- Consistent UI
- Easy future platform expansion

Negative

- Larger application size

6.References

- FR-06
- FR-07
- FR-08
- NFR-10
- NFR-20

ADR-006: Implement JWT Authentication and Role-Based Access Control

Date: 2026-06-11

Status: Accepted

1.Context

Different users require different permissions:

- Owner
- Branch Manager
- Reception Staff
- Kitchen Staff

- Rider

Sensitive business data must be protected.

Relevant Requirements:

- FR-17 Role-Based Access Control
- FR-18 Secure Authentication
- NFR-04 User Authentication
- NFR-05 Password Encryption
- NFR-06 Role-Based Restriction

2.Decision

Use JWT-based authentication with Role-Based Access Control (RBAC).

3.Rationale

JWT provides:

- Stateless authentication
- Secure API access
- Scalability across branches

RBAC ensures users only access authorized features.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
Session-Based Authentication	Easy implementation	Less scalable	Not suitable for distributed architecture
OAuth Only	Very secure	Complex implementation	Unnecessary complexity
JWT + RBAC	Secure and scalable	Token management required	Selected

5.Consequences

Positive

- Improved security
- Supports multi-branch architecture
- Easier permission management

Negative

- Token expiration handling required
- Additional security configuration needed

6.References

- FR-17
- FR-18
- NFR-04
- NFR-05
- NFR-06
- Section 1.2 (JWT Definition) in SRS Document

ADR-007: Use REST API for System Communication

Date: 2026-06-11

Status: Accepted

1.Context

The BK Restaurant Management System consists of multiple client applications, including the React web application and Flutter rider mobile application. These clients need a standardized method to communicate with the backend services.

Relevant Requirements:

- FR-01 Centralized Order Management
- FR-06 Rider Mobile Application
- FR-15 Multi-Branch Monitoring Dashboard
- NFR-16 Modular Architecture

2.Decision

Use RESTful APIs over HTTPS for communication between frontend applications and backend services.

3.Rationale

REST APIs are widely adopted, easy to implement, and supported by both React and Flutter. They provide a standardized communication mechanism and simplify integration between system components.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
GraphQL	Flexible data retrieval and reduces over-fetching	More complex implementation and learning curve	Unnecessary complexity for current project requirements
Direct Database Access	Quick implementation	Security risks and tight coupling between components	Violates modular architecture principles
REST API	Simple, scalable, secure, and widely supported	Requires API development and maintenance	Selected

5.Consequences

Positive

- Clear separation between frontend and backend
- Easier maintenance
- Improved security
- Future integration support

Negative

- Additional API development effort
- Network overhead

6.References

- NFR-16 Modular Architecture
- Section 2.3 Operating Environment in SRS Document

ADR-008: Use Cloud-Based Deployment

Date: 2026-06-11

Status: Accepted

1.Context

The system must support three restaurant branches and provide continuous access to data from different locations.

Relevant Requirements:

- NFR-21 Operating Hours
- NFR-22 Continuous Operation

- NFR-14 Branch Expansion

2.Decision

Deploy the system on a cloud hosting platform such as AWS or Railway.

3.Rationale

Cloud deployment provides centralized access, easier maintenance, higher availability, and support for future expansion without major infrastructure investment.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
Local Server at Restaurant	Full control	Hardware failures, maintenance burden	Poor scalability
Separate Branch Servers	Branch independence	Synchronization complexity	Difficult to manage
Cloud Hosting	Centralized, scalable, reliable	Requires internet connectivity	Selected

5.Consequences

Positive

- Centralized management
- Easier backups
- Supports multiple branches
- Better scalability

Negative

- Internet dependency
- Hosting costs

6.References

- Section 2.3 Hosting Environment in SRS Document
- NFR-21
- NFR-22
- NFR-14

ADR-009: Implement Offline Caching and Synchronization

Date: 2026-06-11

Status: Accepted

1.Context

Some restaurant locations experience unstable internet connectivity, particularly in Ambalangoda and Kahawa.

Relevant Requirements:

- NFR-11 Offline Sync
- NFR-12 Data Consistency
- Section 2.4 Design Constraints

2.Decision

Implement local caching and automatic synchronization mechanisms to temporarily store critical operational data when internet connectivity is unavailable.

3.Rationale

Restaurant operations must continue even during temporary network interruptions. Offline synchronization prevents data loss and supports uninterrupted order processing.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
Always Online System	Simpler implementation	System unusable during outages	Not acceptable
Manual Backup Procedures	Low development effort	High operational burden	Not practical
Offline Cache and Sync	Improves reliability	Additional development effort	Selected

5.Consequences

Positive

- Continuous operation during outages
- Reduced data loss risk
- Improved reliability

Negative

- Synchronization complexity
- Potential conflict resolution requirements

6.References

- NFR-11 Offline Sync
- NFR-12 Data Consistency
- Section 2.4 Design Constraints in SRS Document

ADR-010: Integrate Google Maps GPS Services for Delivery Tracking

Date: 2026-06-11

Status: Accepted

1.Context

The system requires real-time delivery tracking and rider location monitoring.

Relevant Requirements:

- FR-07 GPS-Based Delivery Tracking
- FR-08 Delivery Status Updates
- UC-08 Update Delivery Status
- UC-09 Monitor Delivery Progress

2.Decision

Use Google Maps APIs and smartphone GPS services for rider tracking and route visualization.

3.Rationale

Google Maps provides accurate mapping, route information, location tracking, and navigation services that are well-supported in Flutter applications.

4.Alternatives Considered

Alternative	Pros	Cons	Reason Rejected
OpenStreetMap	Free and open source	Additional configuration required	Less mature support
Custom GPS Solution	Full control	High development effort	Not practical
Google Maps API	Reliable, accurate, well documented	API usage costs	Selected

5.Consequences

Positive

- Accurate rider tracking
- Improved delivery visibility
- Better customer service

Negative

- Dependency on third-party service
- Potential API costs

6.References

- FR-07 GPS-Based Delivery Tracking
- UC-08 Update Delivery Status via Mobile
- UC-09 Monitor Delivery Progress
- Section 2.5 Assumptions and Dependencies in SRS Document